

Deploying an OpenClaw Agent as an Administrative Assistant in Microsoft 365

Materialized update note: This file was reconstructed from the already-completed research update after the prior downloadable artifact failed to persist. For the implementation you requested, the addendum at the end of this document **supersedes older references** in the draft to **Application Access Policies** and any delegated-first patterns. Use **app-only authentication + Exchange Online Application RBAC** as the target design.

Executive summary

An MVP OpenClaw-based administrative assistant in the Microsoft 365 ecosystem is best implemented as an event-driven service that: (a) monitors a user mailbox for “meeting requests” and time-coordination emails, (b) applies deterministic triage and scheduling rules, and (c) replies through a dedicated delegate mailbox using Exchange Online delegation (“Send As” or “Send on behalf”). The most operationally reliable pattern is Microsoft Graph change notifications (subscriptions) feeding a hardened webhook endpoint, with a queue-backed worker that pulls message details and executes deterministic logic. Webhooks reduce polling, while delta queries provide a durable reconciliation mechanism when notifications are delayed, duplicated, or missed.

For authentication/authorization, there are two viable MVP tracks:

- **Enterprise/IT-friendly (recommended for a “real assistant” service): application permissions (app-only)** via OAuth 2.0 client credentials, **restricted to specific mailboxes** using Exchange Online Application RBAC (RBAC for Applications); this avoids refresh-token storage, supports mailbox-wide monitoring and subscriptions, and aligns with least-privilege scoping at the mailbox level.
- **User-centric (faster in a small tenant, but operationally trickier): delegated permissions** via authorization-code flow (with refresh tokens) and a “service account” sign-in, plus Exchange mailbox delegation; workable, but delegate subscriptions for shared/delegated folders have restrictions and will often run into conditional access / operational governance constraints.

A deterministic scheduling MVP should compute availability using **calendar/getSchedule** (free/busy) and rules you control, rather than relying on Graph's **findMeetingTimes** (which is (1) **delegated-only** and (2) explicitly "fine-tuned from time to time," meaning outputs may drift even under static inputs).

Security and compliance hinge on blast-radius control (least privilege, mailbox scoping, strong credential handling), auditability (Graph activity logs + Purview audit), and agent hardening against prompt-injection and tool-abuse patterns common in action-taking agents.

MVP functional design and deterministic operating model

The MVP requirements map cleanly onto a two-stage pipeline: **(1) detection and classification** of inbound emails, followed by **(2) deterministic scheduling and reply composition**.

Detection should handle two input categories:

- **Meeting requests** that arrive as Outlook "event message" types. In Graph, a meeting request arrives in an invitee's mailbox as an **eventMessage** with `meetingMessageType=meetingRequest`; Outlook also automatically creates a tentative calendar event for the invitee.
- **"Asks for time" emails** that may be ordinary messages asking to schedule (no calendar object yet). These can be detected with deterministic rules (subject/body pattern matching) using message fields such as `subject` and `bodyPreview`.

Determinism is easiest if you treat OpenClaw as the "agentic shell" (routing, conversation interface, tool invocation), but keep all "decision authority" in your own rule engine:

- OpenClaw can summarize threads or extract candidate constraints ("next week", "30 minutes", "avoid Friday"), but the final triage priority, time-window selection, and proposed time slots should come from deterministic code paths.
- This reduces risk of "drift" and makes behavior easier to audit and justify (particularly important when acting via delegated mailboxes).

A practical MVP boundary is:

- **Now:** read mail, classify/triage, compute candidate time slots deterministically, send a reply.
- **Next:** create meetings as a delegate by creating events on the user's calendar (with idempotency) once the other party confirms a slot.

Microsoft Graph APIs and endpoints required

This section focuses on Microsoft Graph REST endpoints that directly implement the MVP functions and the “later” delegated-meeting creation capability.

Mail ingestion, meeting-request detection, and thread-safe replies

Core mail retrieval:

- GET `/users/{id|userPrincipalName}/mailFolders/{id}/messages` (or `/me/...`) to list messages in a folder; supported with minimal scopes like **Mail.ReadBasic** (or higher).
- GET `/users/{id}/mailFolders/{id}/messages/delta` to track incremental changes in a folder and maintain a sync cursor (@odata.deltaLink).
- GET `/users/{id}/messages/{message-id}` to fetch full message properties when a candidate is detected (use sparingly; prefer \$select).

Meeting-request representation and extraction:

- Meeting requests can appear as **eventMessageRequest** (derived from eventMessage).
- The **event navigation property** on eventMessage can be used to access the associated calendar event (useful later if you choose to respond programmatically to invites).

Reply mechanisms that preserve conversation/threading:

- POST `/users/{id}/messages/{message-id}/reply` (or `/replyAll`) to reply in a single call (saves to Sent Items).
- POST `/users/{id}/messages/{message-id}/createReply` to create a draft reply you can further edit (add structured content, attachments, headers) and then send.

Sending mail (including from delegated identities):

- POST `/users/{id}/sendMail` (or `/me/sendMail`) sends a composed message; includes `saveToSentItems` behavior.
- To send “from another mailbox” in a delegated-user-token scenario, Graph relies on **Mail.Send.Shared** plus Exchange mailbox permissions, and maps “Send on Behalf” vs “Send As” via sender and from.

Deterministic availability lookup and calendar operations

Free/busy lookup (deterministic input → deterministic output on your side):

- POST `/users/{id}/calendar/getSchedule` (or `/me/calendar/getSchedule`) returns availability blocks for users/resources for a time range. It supports both delegated

and application permissions and explicitly documents least-privileged options like **Calendars.ReadBasic**.

- Use Prefer: outlook.timezone="..." to receive start/end times in a desired time zone.

Mailbox time zone / working hours inputs (needed for deterministic rules):

- GET /users/{id}/mailboxSettings OR GET /users/{id}/mailboxSettings/timeZone returns the user's preferred time zone and supports least-privileged scope **MailboxSettings.Read**.

Meeting creation (later "delegate creates event" capability):

- POST /users/{id}/calendar/events OR POST /users/{id}/calendars/{id}/events creates an event; application permissions are supported (Calendars.ReadWrite) and examples include an optional transactionId for deduplicating retries.

Identity, OAuth flows, permissions, and token management

Delegated vs application permissions

Microsoft Graph supports two access scenarios:

- **Delegated access:** app acts on behalf of a signed-in user; effective permissions are the intersection of app scopes and the user's own rights.
- **App-only access:** app acts as itself (no user), with "application permissions/app roles," and can access any data the permission covers (hence the need for strong scoping controls).

For this assistant MVP, the critical practical differences are:

- Webhook subscriptions for Outlook resources often work best with **application permissions** when monitoring mailboxes beyond a signed-in user context; moreover, delegated shared-folder subscriptions have documented limitations.
- App-only permissions are powerful and typically require **tenant-wide admin consent**; this is explicitly called out as sensitive and should be governed.

OAuth flows you will use

Delegated path:

- **Authorization code flow (with PKCE as appropriate)** is the standard for obtaining tokens "on behalf of a user," and supports refresh mechanisms. Microsoft recommends using supported auth libraries rather than hand-crafting protocol calls.

- Requesting long-lived access typically involves `offline_access` (and secure refresh-token storage), and refresh-token behavior is documented in the identity platform refresh-token guidance.

App-only path (recommended for the MVP service backend):

- **OAuth 2.0 client credentials flow** is explicitly designed for daemon/service-to-service workloads running in the background with no user interaction.
- In client-credentials scenarios, tokens are obtained per service need and rotated by re-acquiring; `.default` scope patterns are the standard way to request statically-consented application permissions.

Token caching and refresh strategies

For production reliability, use MSAL token caching (even in app-only flows) to reduce token endpoint traffic and improve resilience:

- MSAL maintains token caches and supports “acquire silently” patterns; for confidential clients, you typically manage cache serialization per user/session if you run delegated flows at scale.
- If you use delegated tokens, you must design secure persistence for refresh tokens (encrypted at rest, access-controlled) and plan for invalidation (password reset, conditional access changes, consent revocation).

Credential storage best practice:

- Prefer certificate-based client authentication and store certificates as Key Vault certificate objects (not as secrets), with least-privileged access controls and rotation.

Multi-tenant considerations:

- You can register apps as single-tenant or multitenant; multitenant configurations are explicitly documented and determine who can sign in/consent.
- Microsoft Graph activity logs are tenant-scoped; they help tenant admins see Graph request trails in their own tenant, but do not give you observability into “another tenant” for a multitenant app. This matters if you are building a SaaS assistant.

Mailbox and calendar delegation design

Mailbox delegation and “Send As” vs “Send on behalf”

Exchange Online defines three relevant delegate concepts:

- **Full Access:** delegate can open/manage mailbox contents but cannot send as that mailbox.

- **Send As:** messages appear to come from the mailbox; if both “Send As” and “Send on behalf” exist, Send As typically takes precedence.
- **Send on behalf:** recipients see “ on behalf of .” In Graph message objects, this is modeled via sender (actual sender) and from (apparent author).

Operational setup:

- Delegation is typically configured in the Exchange admin center or Exchange Online PowerShell. For example, Exchange Online docs describe using `Add-RecipientPermission... -AccessRights SendAs` and “`GrantSendOnBehalfTo`” on mailbox/group cmdlets.

Graph dependency:

- In delegated-token scenarios, Graph requires **Mail.Send.Shared**, and the signed-in user must have the corresponding Exchange mailbox permission(s). Graph explicitly notes it cannot query which mailboxes the user has delegated rights for—so you must maintain your own configuration map (e.g., allowed From addresses per delegate account).

Dedicated delegate mailbox patterns

Your requirement says replies should come from a dedicated Office 365 delegate account (Send As or Send on behalf). In practice, there are three patterns:

- **Delegate account as the visible sender (simple):** send from the delegate mailbox (no impersonation), using `POST /users/{delegate}/sendMail`. This does not require SendAs rights, but it does change recipient expectations (they see the assistant address).
- **Delegate account sending “on behalf of” the user:** configure “Send on behalf” so the delegate is visible as delegate; Graph maps this using `from` and `sender`.
- **Delegate account sending “as the user” (strongest illusion, highest risk):** configure Send As so email appears to come from user; the assistant identity is not visible to recipients. This can be desirable for UX but increases governance risk; ensure auditing and explicit policy controls.

Calendar delegation and “delegate creates events”

Microsoft’s calendar-sharing model distinguishes between “sharing” and “delegation.” Delegates can have write access and can respond to meeting requests on behalf of the owner.

Graph supports calendar permission objects:

- Creating a calendar permission (share/delegate) is done via the calendar permissions APIs (calendarPermission resource).

- A key nuance: listing/creating/updating calendar permissions is “supported on behalf of only the calendar owner”; if you try to get permissions “as a delegate,” the collection can be empty. This affects how you build admin tooling.

Creating the meeting:

- To create an event in the user’s calendar, use the “create event” endpoints with **Calendars.ReadWrite** (delegated or application). This operation supports a client-specified `transactionId` to avoid redundant POSTs during retries/timeouts, which is highly relevant for resilient delegates.

Eventing, synchronization, and error handling

Webhooks and subscriptions

Microsoft Graph change notifications allow event-driven handling instead of polling. The subscription model is:

1. create subscription (POST /subscriptions),
2. endpoint is validated via a `validationToken`,
3. Graph sends notifications to your `notificationUrl` while subscription is valid.

Webhook validation requirements are strict:

- Graph calls your endpoint with `validationToken` and expects **HTTP 200**, `text/plain`, and the decoded token returned **within 10 seconds**.
- Microsoft explicitly advises treating validation tokens as opaque and recommends escaping HTML/JS to reduce XSS-style risks (even though Graph won’t send HTML/JS).

Subscription lifetimes and renewal:

- Outlook message/event/contact subscriptions have maximum expiration of **10,080 minutes (~7 days)**; subscriptions with resource data (“rich notifications”) have a **1,440 minute (~1 day)** lifetime.
- Outlook resources also have a **maximum of 1,000 active subscriptions per mailbox for all applications** (design to reuse subscriptions; avoid per-folder explosions).
- Lifecycle notifications exist to reduce missed notifications (`reauthorizationRequired`, `removed`, `missed`). Implementing lifecycle handlers is part of building a reliable assistant.

Practical implications for your MVP:

- You should run a **subscription-renewal job** (e.g., every few hours/day) and store subscription metadata (IDs, expiration, `clientState`) in durable storage.

- Your webhook handler should be “thin”: validate/ack quickly, enqueue work, return success; do heavy logic in a worker to avoid timeout risk.

Delta queries as the “safety net”

Delta query is the canonical way to synchronize changes and recover from notification gaps:

- message: delta provides incremental message changes in a folder and returns delta links you persist for later rounds.
- You should treat webhooks as “wake signals” and delta queries as the authoritative reconciliation mechanism (especially when notifications are duplicated or missed).

Retry, throttling, and idempotency

Throttling and retries:

- Microsoft Graph uses HTTP **429** with Retry-After; the recommended recovery is to wait per Retry-After and retry; if missing, use exponential backoff. SDKs often implement these handlers.
- Design your worker to be queue-driven with visibility timeouts, retry counts, and dead-lettering for poison messages.

Idempotency patterns:

- For **event creation**, use event transactionId so that retries don’t create duplicate meetings when the client times out or is retried by queue infrastructure.
- For **mail replies**, maintain an internal dedupe layer keyed by (tenantId, mailboxId, messageId/immutableId, actionType) because sendMail and reply are not inherently idempotent. Also note Microsoft’s warning not to assume message IDs remain stable after moves/copies; consider immutable IDs where appropriate.

Security and compliance considerations

Least privilege and blast-radius control

The assistant touches highly sensitive resources (mail + calendar). Key controls:

- Prefer minimal Graph permissions: the list-messages and delta APIs support “least privileged” permissions such as **Mail.ReadBasic** or **Mail.ReadBasic.All** for app-only (with higher scopes only if you must read bodies/attachments).
- For free/busy, getSchedule supports **Calendars.ReadBasic** as least privileged, reducing exposure compared to full event read.
- App-only permissions for Exchange data sets (mail/calendars/contacts/mailbox settings) can reach all mailboxes by default; for the implementation requested in

this document, the mitigation should be Exchange Online **Application RBAC (RBAC for Applications)** with a resource scope that contains only the allowed mailboxes.

Consent, admin consent, and governance

- Application permissions require admin consent; Microsoft explicitly warns that granting tenant-wide admin consent is sensitive because it can expose large portions of organizational data unless restricted.
- Maintain a formal permission review, and pin permissions to MVP scope only.

Audit logging and monitoring

For forensic readiness and operational confidence:

- Microsoft Graph activity logs provide an audit trail of HTTP requests Graph receives/processes for a tenant and can be streamed via Azure Monitor diagnostic settings to Log Analytics/Storage/Event Hubs.
- Microsoft Purview auditing solutions provide a unified audit log capturing many user/admin operations across Microsoft services, supporting investigations and compliance obligations.

Data residency and retention boundaries

- Exchange Online mailbox geography and data residency are tenant properties (and can be discovered/administered); mailbox location and regional properties are documented in Exchange Online data residency guidance.
- Align your agent's data stores with your tenant's compliance posture: store only necessary metadata (message ID, sender, derived classification, chosen time slots) rather than full message bodies, unless required.

Agent-specific hardening for OpenClaw

OpenClaw positions itself as an assistant that can “clear your inbox” and “manage your calendar,” so it's inherently an action-taking agent with significant blast radius if misconfigured.

Two high-priority controls for OpenClaw-style agent frameworks:

- **Input-to-tool execution control:** treat email bodies and external content as untrusted inputs that can influence tool execution; Microsoft's security guidance for OpenClaw emphasizes inventorying deployments, verifying identities/permissions used, and identifying which inputs can influence tool execution to reduce blast radius.
- **Skill/plugin supply chain controls:** OpenClaw's own security program highlights that skills run as code “in your agent's context” with access to tools/

data, and it has added malware-scanning layers (VirusTotal) for skill marketplace content—useful, but not a complete guarantee. For enterprise deployments, pin to a vetted allowlist of internal skills and disable arbitrary third-party skill execution.

Deployment options, reference architecture, and MVP plan

Recommended architecture

The architecture below is designed to be deterministic, auditable, and resilient (queue-backed), while still compatible with OpenClaw as the orchestration layer.

```
flowchart LR
    subgraph M365[Microsoft 365]
        UMailbox[User mailbox: Inbox + Calendar]
        DMailbox[Delegate mailbox: Send-As / Send-on-behalf]
        Graph[Microsoft Graph]
    end

    subgraph Azure[Azure-hosted MVP]
        Webhook[Webhook handler\n(validateToken + enqueue)]
        Queue[Queue\n(Service Bus / Storage Queue)]
        Worker[Deterministic worker\n(classify + schedule + reply)]
        Store[State store\n(subscriptions, deltaLinks, dedupe)]
        Vault[Key Vault\n(cert/secret)]
    end

    Graph -->|Change notifications| Webhook
    Webhook --> Queue
    Queue --> Worker
    Worker --> Store
    Worker -->|GET messages/delta| Graph
    Worker -->|getSchedule| Graph
    Worker -->|sendMail/reply| Graph
    Vault --> Worker
    UMailbox --> Graph
    DMailbox --> Graph
```

This aligns with Graph’s webhook validation constraints (fast response) and supports durable processing with retries/backoff.

Deterministic triage rule examples and pseudocode

A deterministic triage model should avoid probabilistic scoring from the LLM. Typical rule inputs you can obtain deterministically from Graph message metadata include: internal vs external sender domain, known VIP allowlist, mail importance flag, presence of deadlines, and meeting-request type.

Example triage rules (illustrative):

- **Priority 0 (immediate):** sender is in VIP_EMAILS, or subject contains “urgent” plus sender is internal.
- **Priority 1:** meeting request (meetingMessageType=meetingRequest) from internal domain.
- **Priority 2:** “ask for time” pattern + internal sender.
- **Priority 3:** external sender not in allowlist; propose fewer slots and require confirmation.

Pseudocode:

```
function triage(message):
    sender = normalize(message.from.emailAddress.address)
    subject = normalize(message.subject)
    preview = normalize(message.bodyPreview)

    isMeetingReq = (message.odataType contains "eventMessage") and
                    (message.meetingMessageType == "meetingRequest")

    isTimeAsk = regex_match(subject + " " + preview,
                             r"(what time works|can we meet|availability|free (this|next) week|
                             schedule.*call)")

    if sender in VIP_EMAILS:
        return P0

    if isMeetingReq and sender_domain(sender) == INTERNAL_DOMAIN:
        return P1

    if isTimeAsk and sender_domain(sender) == INTERNAL_DOMAIN:
        return P2

    if isTimeAsk:
        return P3

    return IGNORE
```

Meeting-time selection should be deterministic and driven by getSchedule plus a policy table: working hours, minimum notice, preferred meeting lengths, “no-meeting blocks,” and time-zone normalization using mailbox settings.

Deterministic availability algorithm sketch:

```

function propose_times(request, userMailboxId):
    tz = graph.get("/users/{userMailboxId}/mailboxSettings/timeZone")
    workingHours = graph.get("/users/{userMailboxId}/mailboxSettings/workingHours")

    window = compute_window(request, workingHours, tz) // e.g., next 5 business
days, 9-17
    freeBusy = graph.post("/users/{userMailboxId}/calendar/getSchedule", window)

    candidateSlots = []
    for day in window.days:
        for slot in day.slots(step=30min):
            if slot inside workingHours AND slot not in NO_MEETING_BLOCKS
            AND freeBusy(slot) == "free"
            AND slot starts >= now + MIN_NOTICE:
                candidateSlots.append(slot)

    // deterministic ranking: earliest-first, then preference (e.g., Tue-Thu > Mon/
Fri)
    candidateSlots = sort(candidateSlots, by=[dayPreference, startTime])

    return first N slots (e.g., 3-5), formatted in tz

```

End-to-end flow example

```

sequenceDiagram
    participant Graph as Microsoft Graph
    participant WH as Webhook handler
    participant Q as Queue
    participant W as Worker
    participant EX as Exchange Online (mail/calendar)

    EX->>Graph: New message arrives in Inbox
    Graph->>WH: POST notificationUrl (change notification)
    WH->>Graph: 202 Accepted (after quick validation/ack)
    WH->>Q: Enqueue {userId, messageId, changeType}

    Q->>W: Dequeue work item
    W->>Graph: GET message (minimal $select) / or delta sync
    W->>Graph: POST calendar/getSchedule (free/busy)
    W->>Graph: POST reply or sendMail (from delegate identity)
    W->>Q: Ack + persist dedupe/state

```

This combines push (notifications) with pull (delta/message fetch) for correctness.

Implementation options comparison

Option	Strengths	Constraints / risks	Best fit for MVP
Azure Functions (HTTP webhook + queue-trigger worker)	Designed for event-driven scaling; easy to separate webhook and worker; strong ecosystem for queues/monitoring	Cold-start and runtime constraints must be managed; more “code” to build than low-code options	Recommended (flexible, deterministic logic, easy webhook compliance)
Azure Container Apps (webhook API + background worker)	Fully managed “serverless containers,” automatic scaling, good for long-running workers and custom runtimes	Slightly more ops surface than Functions; need container build/pipeline	Strong if OpenClaw runtime is containerized already
Azure Logic Apps	Low-code orchestration with Azure-native integration, DevOps + monitoring advantages over Power Automate	Complex deterministic logic/policies can get unwieldy; webhook validation and high-control auth flows may require custom code anyway	Good as orchestrator, but often paired with a code webhook
Power Automate	Fastest citizen-dev path inside Microsoft 365; many connectors	Less control over auth nuances, deterministic policy engines, and webhook validation behaviors; licensing and scale constraints	Best for prototypes, not for a hardened “assistant agent” backend

Prioritized step-by-step MVP plan with effort and risks

The effort estimates below assume one experienced engineer with admin access to the tenant, plus occasional IT/security review.

Build foundation (2–4 days)

- Register an app in Entra ID; choose **single-tenant** for MVP unless you are explicitly building SaaS.

- Decide permission model: **app-only + Exchange Online Application RBAC (RBAC for Applications)** (recommended) vs delegated fallback. Configure the app registration and Exchange role assignments accordingly.
- Create a dedicated delegate mailbox (licensed or shared mailbox, per org policy) and configure Send As / Send on behalf delegation using Exchange admin center or PowerShell.

Risks: admin consent approvals can take time; delegation settings can be misunderstood (Send As vs on behalf precedence).

Implement ingestion + eventing (3–6 days)

- Build webhook endpoint that passes Graph validation token requirements and quickly enqueues work.
- Create message subscriptions to the relevant inbox folder(s) and implement renewal logic respecting subscription lifetimes.
- Add delta-query reconciliation for the inbox folder (messages/delta) and persist @odata.deltaLink.

Risks: subscription renewal bugs cause silent degradation; webhook endpoint must be internet-reachable and fast.

Deterministic detection + triage (2–4 days)

- Implement meeting-request detection via eventMessage semantics (meetingMessageType) and “ask for time” regex rules.
- Implement deterministic priority rules and routing/queuing.
- Add a configuration store for VIP lists, internal domains, allowed external senders.

Risks: false positives/negatives in pattern matching; need controlled rollout.

Deterministic scheduling + reply (4–7 days)

- Implement getSchedule calls and deterministic slot selection respecting mailbox time zone/working hours.
- Send replies using /reply or /createReply then send; ensure send uses the delegate mailbox identity and (if required) Send As / on-behalf semantics.
- Implement throttling-safe retries (429 + Retry-After).

Risks: time-zone edge cases; schedule “truth” vs user expectations; throttling under bursts.

Operational hardening (2–5 days)

- Turn on Graph activity logs to Log Analytics and set alerts for anomalous call volume or privileged operations.

- Add Purview audit review procedures for mailbox delegation changes and agent actions.
- Apply defensive controls for OpenClaw runtime: strict skill allowlist, tool execution constraints, and monitored identity/permissions.

Risks: without strong controls, action-taking agents create governance risk; skill supply-chain risks require defense-in-depth.

Enable “create events as delegate” later (3–6 days)

- Add a confirmation step (explicit “yes” from user or deterministic confirmation email from requester).
- Create events using `POST /users/{id}/calendar/events` with `transactionId` and send invitations.
- Optionally, manage calendar delegation via `calendarPermission` APIs (noting owner-only management semantics).

Risks: mistaken bookings; duplicate meetings without `transactionId`; additional compliance review may be required for automatic invites.

Addendum: App-only + mailbox-scoped Exchange Online authorization via Application RBAC

This addendum is the implementation-specific update for your requested design. It changes the earlier draft in four important ways:

1. It replaces earlier references to **Application Access Policies** with **Exchange Online RBAC for Applications (Application RBAC)** as the mailbox-scoping mechanism.
2. It makes **reading full meeting contents** part of the MVP, not just free/busy.
3. It defines a clear **rescheduling pathway** for both organizer-owned meetings and attendee-side “propose a new time” flows.
4. It adds two concrete execution sections: **Actions For Administrator** and **Actions for Developer**.

Recommended target design

Use the following model for the production-facing MVP:

- **Authentication:** OAuth 2.0 client credentials (app-only).
- **Authorization:** Exchange Online **RBAC for Applications**.

- **Mailbox scope:** a Management Scope or Administrative Unit that contains only the mailboxes the agent is allowed to touch.
- **Mail and calendar protocol:** Microsoft Graph.
- **Assistant sender identity:** an in-scope assistant mailbox that the app can read and send from using Application RBAC.
- **MVP behavior:** monitor the scoped mailbox(es), read meeting content, compute availability deterministically, and send responses.
- **Roadmap behavior:** reschedule meetings when allowed by role assignment and business rules.

Why this is the right authorization model for this version

Application RBAC is the Exchange Online authorization model that lets you bind an app to a **specific Exchange application role** and a **specific mailbox scope**. For this project, that is the right fit because it lets you keep the service **app-only**, while still limiting it to a mailbox subset. It also replaces Application Access Policies for this use case.

A critical implementation detail: **Microsoft Entra application grants and Exchange Application RBAC grants are additive**. If the app already has broad Microsoft Graph application permissions like Mail.Read, Mail.Send, Calendars.Read, Calendars.ReadWrite, or MailboxSettings.Read consented tenant-wide in Entra, those unscoped grants can defeat the mailbox scoping you intended to enforce with Application RBAC. Treat that as an explicit admin review item.

Minimum role set for your requested MVP

Use the smallest Exchange Application RBAC role set that supports the requested behavior:

- **Application Mail.Read** Needed if the assistant will monitor and read coordination mail, inbox threads, and message change notifications.
- **Application Calendars.Read** Needed to read meeting subject, attendees, organizer, body, location, online meeting metadata, and other event properties.
- **Application MailboxSettings.Read** Needed to read time zone and working hours so scheduling logic is deterministic and timezone-safe.
- **Application Mail.Send** Needed if the assistant sends replies from a scoped assistant mailbox using app-only auth.

Add the following when you are ready to enable write behavior:

- **Application Calendars.ReadWrite** Needed to update organizer-owned events and to support the attendee-side rescheduling path where the agent proposes a new time.

Do **not** use broad compound roles such as Application Exchange Full Access or Application Mail Full Access for this MVP unless you intentionally want a much larger blast radius.

Reading meeting contents in the MVP

The earlier draft focused heavily on mail ingestion and availability. For the MVP you requested, the assistant should also read the **actual meeting content**. In practice, that means reading **event objects**, not just free/busy blocks.

Recommended read pattern:

1. **Use mailbox notifications to detect relevant activity.** Message subscriptions are still useful if the assistant begins from inbox activity such as meeting requests or scheduling emails.
2. **Resolve to calendar data for the actual meeting content.** Use calendar APIs to read:
 - subject
 - body / body preview
 - organizer
 - attendees
 - start / end
 - location
 - online meeting info
 - recurrence / occurrence data

3. Preferred API pattern

- For time-window reads: GET /users/{id|userPrincipalName}/calendarView?startDateTime=...&endDateTime=...
- For direct event reads: GET /users/{id|userPrincipalName}/events/{event-id}

Implementation notes:

- calendarView is the better first-class MVP read surface because it can return occurrences, exceptions, and single instances for a time window.
- calendarView supports Prefer: outlook.body-content-type="text" (or "html"), so you can retrieve the body in a format that is easier for downstream processing.
- GET /events/{id} returns the event body in HTML, so if you use the single-event API and want plain text for the agent, normalize HTML to text in your code.
- Keep getSchedule for deterministic availability, but do **not** treat it as the source of meeting content.

Pathway to rescheduling meetings

Rescheduling is feasible, but the implementation path depends on whether the mailbox owner is the **organizer** or merely an **attendee**.

Case 1: mailbox owner is the organizer

This is the cleanest write path.

Use:

- PATCH /users/{id|userPrincipalName}/events/{event-id}

Update at least:

- start
- end

You may also update:

- subject
- body
- location
- attendees

Important guardrail:

- If the event is an online meeting and you update the body, preserve the online meeting blob. Removing it can disable the online meeting metadata.

This organizer-owned flow is the main reason to stage or enable **Application Calendars.ReadWrite**.

Case 2: mailbox owner is an attendee

When the mailbox owner is **not** the organizer, the agent should not silently rewrite the organizer's event. Instead, use the Outlook meeting-proposal flow.

Use:

- POST /users/{id|userPrincipalName}/events/{event-id}/tentativelyAccept

with:

- sendResponse = true
- proposedNewTime = { start, end }

Constraints:

- The event must allow new time proposals (allowNewTimeProposals = true).
- This is a proposal flow, not a forced organizer rewrite.

- The organizer still decides whether to accept the proposed time.

Recommended product stance:

- Keep attendee-side “propose a new time” behind a feature flag initially.
- Enable organizer-owned PATCH updates first.
- Add attendee-side meeting proposals after you have clear business rules and audit trails.

Practical architecture adjustment to the earlier draft

Keep the earlier event-driven architecture (subscriptions → webhook → queue → worker), but change the functional contract to:

1. **Detect relevant inbox or calendar activity.**
2. **Read the meeting/event details from the calendar, not only from email text.**
3. **Read mailbox settings for timezone and working hours.**
4. **Use deterministic scheduling logic.**
5. **Send the reply from an in-scope assistant mailbox.**
6. **Only enable calendar writes after the Application Calendars.ReadWrite role is assigned and your feature flag is turned on.**

Actions For Administrator

This section is the concrete administrator checklist. It is written so your Exchange / Entra administrator can execute it directly.

1) Hold the right admin roles before making changes

For the Exchange Application RBAC work, the administrator should have:

- **Organization Management** in Exchange Online, because that role group can assign the Application RBAC roles.
- **Exchange Administrator** in Microsoft Entra ID, because the documented setup expects that role for assigning these permissions.

If a separate identity team manages the app registration or admin consent workflow, coordinate with them before starting.

2) Create or identify the app registration and service principal

The admin should either create a new app registration for this assistant or designate an existing one that will be used **only** for this assistant workflow.

The admin should record and give you:

- **Tenant ID**

- **Client / Application ID**
- **Enterprise Application service principal Object ID** This is the service principal object in the tenant. It is **not** the App Registration object ID.

Preferred auth material:

- **Certificate-based auth** is preferred for production.
- If they insist on a client secret for the MVP, the secret must be stored in your secret manager and rotated.

What the admin must give you for authentication:

- Tenant ID
- Client ID
- Either:
 - access to the private key for the certificate used by the app, **or**
 - a client secret
- The exact expiration date for that secret or certificate

3) Decide the mailbox scope

The admin must decide which mailboxes the agent is allowed to access. There are two good patterns.

Option A — Management Scope (recommended when you want a mailbox list or group-based scope)

Use a mail-enabled group or another filterable mailbox property to define the scope.

If the admin wants to reuse a scoping group, the most direct pattern is a Management Scope based on MemberOfGroup.

Important limitation:

- Only **direct** group membership counts.
- Nested members are **not** considered in scope.

Typical sequence:

```
Connect-ExchangeOnline
```

```
# Inspect the scoping group and capture its DistinguishedName (DN)
Get-Group -Identity "OpenClaw Scoped Mailboxes" | Format-List
    Name,DistinguishedName

# Create a management scope based on direct membership in that group
New-ManagementScope `
    -Name "OpenClaw-ScopedMailboxes" `
    -RecipientRestrictionFilter "MemberOfGroup -eq '<GROUP_DISTINGUISHED_NAME>'"
```

Use this when you want explicit control over which mailboxes are in scope.

Option B — Administrative Unit

Use this when the organization already manages mailbox ownership and boundaries through a Microsoft Entra Administrative Unit.

In that case, the admin does **not** need a separate Management Scope object. They can assign the role directly with:

- -RecipientAdministrativeUnitScope <ADMIN_UNIT_GUID>

4) Create the Exchange Online service-principal pointer

After the Entra service principal exists, the admin must create the Exchange Online pointer object.

```
Connect-ExchangeOnline
```

```
New-ServicePrincipal `
    -AppId "<CLIENT_APPLICATION_ID>" `
    -ObjectId "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
    -DisplayName "OpenClaw Assistant"
```

Important:

- Use the **Enterprise Application service principal Object ID** from Entra.
- Do **not** use the App Registration object ID.

5) Assign the minimum Exchange Application RBAC roles

For your requested MVP, the admin should assign these roles to the app within the chosen mailbox scope.

Read + send MVP

```
# Mail read
New-ManagementRoleAssignment `
  -Name "OpenClaw-MailRead" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application Mail.Read" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"

# Calendar read
New-ManagementRoleAssignment `
  -Name "OpenClaw-CalendarsRead" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application Calendars.Read" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"

# Mailbox settings read
New-ManagementRoleAssignment `
  -Name "OpenClaw-MailboxSettingsRead" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application MailboxSettings.Read" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"

# Mail send
New-ManagementRoleAssignment `
  -Name "OpenClaw-MailSend" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application Mail.Send" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"
```

Add later when rescheduling is enabled

```
New-ManagementRoleAssignment `
  -Name "OpenClaw-CalendarsReadWrite" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application Calendars.ReadWrite" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"
```

If the admin is using an Administrative Unit instead of a Management Scope, replace -CustomResourceScope... with:

```
-RecipientAdministrativeUnitScope "<ADMIN_UNIT_GUID>"
```

6) Review and remove overlapping unscoped Entra application permissions

This is the most important scoping safeguard.

If the app already has tenant-wide Microsoft Graph **application permissions** such as:

- Mail.Read
- Mail.Send
- Calendars.Read
- Calendars.ReadWrite
- MailboxSettings.Read

the admin should explicitly review them and remove overlapping broad grants before relying on Application RBAC scoping for those same data sets.

What you should ask the admin for here:

- Written confirmation that they checked the Enterprise Application's **Admin consent** list.
- A list of any Graph application permissions that remain.
- Confirmation that no broad overlapping Exchange-data application permissions were left in place unintentionally.

7) Put the sender mailbox inside the scope

If the assistant will send mail from a dedicated assistant mailbox, that mailbox must be **inside the RBAC scope**.

The admin should give you:

- the assistant mailbox UPN / SMTP address
- confirmation that the mailbox is included in the Management Scope or Administrative Unit
- confirmation that Application Mail.Send is assigned for the app within that scope

If the assistant might later create or reschedule meetings on behalf of that mailbox, that mailbox also needs to remain in scope for Application Calendars.ReadWrite.

8) Test the scope before handing the environment to development

The admin should test both an allowed mailbox and a disallowed mailbox.

```
# In-scope mailbox
Test-ServicePrincipalAuthorization `
  -Identity "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Resource "in-scope-user@contoso.com"

# Out-of-scope mailbox
Test-ServicePrincipalAuthorization `
  -Identity "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Resource "out-of-scope-user@contoso.com"
```

What you want to see:

- **In-scope mailbox:** the assigned roles appear and InScope = True
- **Out-of-scope mailbox:** either no effective role or InScope = False

Also note:

- RBAC permission propagation can take **30 minutes to 2 hours** outside the test cmdlet.
- The test cmdlet bypasses that cache and is the fastest way to validate the config.

9) Hand these exact items to the developer

Ask the administrator to give you this package:

- Tenant ID
- Client / Application ID
- Enterprise Application service principal Object ID
- Secret or certificate access details
- The name of the Exchange Management Scope **or** the Administrative Unit GUID
- The scoped mailbox list (or the source group / AU that defines it)
- The assistant mailbox SMTP / UPN that the agent should send from
- Confirmation that these roles are assigned:
 - Application Mail.Read
 - Application Calendars.Read
 - Application MailboxSettings.Read
 - Application Mail.Send
 - Application Calendars.ReadWrite (either enabled now or explicitly deferred)
- One known **in-scope** mailbox for testing
- One known **out-of-scope** mailbox for negative testing
- Confirmation that overlapping broad Entra Graph app permissions were reviewed and removed where necessary

Actions for Developer

This is the step-by-step implementation plan for you.

1) Capture the admin handoff as configuration, not tribal knowledge

Before writing code, record these values in your environment configuration:

- TENANT_ID
- CLIENT_ID
- certificate thumbprint / certificate path / key identifier **or** client secret
- ASSISTANT_MAILBOX_UPN
- at least one IN_SCOPE_TEST_MAILBOX
- at least one OUT_OF_SCOPE_TEST_MAILBOX
- a flag for whether Calendars.ReadWrite is already enabled

Do not start coding against a mailbox until you have both a positive test mailbox and a negative test mailbox.

2) Implement confidential-client authentication

Use a confidential client and the OAuth client credentials flow.

Your runtime should:

1. acquire an app-only token for Microsoft Graph
2. cache tokens in memory (or in a secure shared cache if needed)
3. fail closed if the app credential is missing or expired

Treat the app identity as a production credential:

- keep it in Key Vault or your equivalent
- never bake secrets into code or container images
- prefer certificate auth to client secrets

3) Verify the Application RBAC boundary before building product logic

Your first automated smoke test should prove the scope boundary.

At startup or in a dedicated validation command:

- call a harmless read against an **in-scope** mailbox
- call the same kind of read against an **out-of-scope** mailbox
- assert that the in-scope call succeeds and the out-of-scope call fails

Do this before implementing business logic, so you know you are building on a correctly-scoped authorization model.

4) Build the mailbox/event ingestion path

You have two workable MVP patterns. Use the first one unless you have a strong reason not to.

Preferred MVP ingestion pattern

- Subscribe to mailbox/message changes so the assistant wakes up on inbound coordination activity.
- For any relevant mail item, resolve into calendar data and read the actual meeting content from the event layer.

Why this is the better MVP pattern:

- it preserves the inbox-driven workflow from the original draft
- it lets you detect normal coordination emails and formal meeting requests
- it still gives you access to the actual meeting body and attendees from calendar APIs

To create the subscription, implement:

- POST /subscriptions

For message subscriptions, your app needs read permission to the resource. Since your target design uses app-only Exchange access, treat Application Mail.Read as required for message-driven subscriptions.

Implementation rules:

- keep the webhook handler thin
- validate quickly
- enqueue work
- renew subscriptions on a schedule
- keep a durable table of subscription IDs, expiration times, client state, and target mailboxes

5) Read meeting contents from the calendar layer

For the actual meeting read path, implement these APIs:

Windowed read

```
GET /users/{userPrincipalName}/calendarView?  
startDateTime=2026-04-01T00:00:00Z&endDateTime=2026-04-08T00:00:00Z  
Prefer: outlook.timezone="America/Chicago"  
Prefer: outlook.body-content-type="text"
```

Use this when:

- you need upcoming meetings
- you need occurrences of recurring meetings
- you want a windowed availability + content view

Direct event read

```
GET /users/{userPrincipalName}/events/{eventId}  
Prefer: outlook.timezone="America/Chicago"
```

Use this when:

- you already know the event ID
- you need to hydrate a specific meeting from a queue item

Implementation note:

- event bodies from direct GET /events/{id} come back in HTML, so normalize them in code if the agent needs plain text.

Suggested minimum fields to store after normalization:

- event ID
- iCalUid
- subject
- body text
- organizer
- attendee list
- start / end + timezone
- location
- isOnlineMeeting / onlineMeetingProvider
- allowNewTimeProposals
- lastModifiedDateTime

6) Read mailbox settings for scheduling context

Before proposing or modifying times, read mailbox settings:

```
GET /users/{userPrincipalName}/mailboxSettings
```

Use this to pull:

- time zone

- working hours
- locale if needed for formatting

Do not hard-code timezone assumptions in the scheduler.

7) Keep availability logic deterministic

For the scheduling engine itself, continue using deterministic logic and `getSchedule` / free-busy style availability, rather than relying on probabilistic meeting recommendation APIs.

The implementation contract should be:

- LLM / OpenClaw may summarize intent and constraints
- deterministic code computes candidate slots
- policy config decides minimum notice, business hours, meeting lengths, blackout windows, and ranking order
- the final send action is driven by code, not model improvisation

8) Implement send behavior using the scoped assistant mailbox

Use app-only send mail:

```
POST /users/{assistantMailboxUpn}/sendMail
```

Prerequisites:

- the assistant mailbox is inside the RBAC scope
- the app has `Application Mail.Send` in that scope

Developer safeguards:

- hard-code an allowlist of mailbox identities the app may send from
- do not allow arbitrary caller-supplied `from` mailboxes
- log every outbound send with message correlation IDs

9) Implement read-only MVP first

Before enabling any write path, complete a read-only milestone:

- detect inbound scheduling messages
- read meeting contents from calendar events
- read mailbox settings
- compute candidate slots
- draft and send a reply from the assistant mailbox
- verify auditing and negative-scope tests

This is the point where the MVP is already useful.

10) Add organizer-side rescheduling behind a feature flag

When the admin enables `Application Calendars.ReadWrite`, add a feature flag such as:

- `ENABLE_ORGANIZER_RESCHEDULE=true`

Implementation path:

1. read the current event
2. verify the mailbox owner is the organizer (or that your business rule allows the edit)
3. patch only the fields you intend to change
4. update start and end
5. if editing body on an online meeting, preserve the meeting blob
6. log the change and the old/new times

Example operation:

```
PATCH /users/{userPrincipalName}/events/{eventId}
Content-Type: application/json

{
  "start": {
    "dateTime": "2026-04-09T14:00:00",
    "timeZone": "America/Chicago"
  },
  "end": {
    "dateTime": "2026-04-09T14:30:00",
    "timeZone": "America/Chicago"
  }
}
```

11) Add attendee-side “propose a new time” as the second write feature

Create a second feature flag:

- `ENABLE_ATTENDEE_PROPOSE_NEW_TIME=true`

Only use this flow when:

- the mailbox owner is **not** the organizer
- the event allows new time proposals
- your product policy says the agent may respond on the user’s behalf

Operation:

```
POST /users/{userPrincipalName}/events/{eventId}/tentativelyAccept
Content-Type: application/json
```

```
{
  "comment": "Tentatively accepting and proposing a different time.",
  "sendResponse": true,
  "proposedNewTime": {
    "start": {
      "dateTime": "2026-04-10T15:00:00",
      "timeZone": "America/Chicago"
    },
    "end": {
      "dateTime": "2026-04-10T15:30:00",
      "timeZone": "America/Chicago"
    }
  }
}
```

Behavioral rule:

- organizer-owned reschedule is a **calendar write**
- attendee-side reschedule is a **meeting proposal**

Do not collapse those two concepts into the same code path.

12) Add audit, idempotency, and kill switches before broad rollout

Before widening the scope beyond a test mailbox set, add:

- idempotency keys for send and update operations
- per-mailbox feature flags
- a global kill switch for outbound sends
- a global kill switch for calendar writes
- structured logs for:
 - target mailbox
 - event ID
 - original times
 - proposed / new times
 - acting feature flag
 - correlation ID
 - result code

13) Suggested developer rollout order

Use this rollout order:

1. auth + scope validation
2. message subscription + webhook
3. calendar content read
4. mailbox settings read
5. deterministic slot computation
6. send from assistant mailbox
7. read-only pilot
8. organizer reschedule
9. attendee propose-new-time
10. broaden mailbox scope only after audit review

Source basis for this addendum

This addendum was materialized from the prior research update and tightened against Microsoft Learn pages covering:

- Exchange Online **RBAC for Applications**
- New-ServicePrincipal
- New-ManagementRoleAssignment
- Test-ServicePrincipalAuthorization
- Microsoft Graph calendarView
- Microsoft Graph event get
- Microsoft Graph event update
- Microsoft Graph event tentativelyAccept
- Microsoft Graph create subscription
- Microsoft Graph get mailboxSettings
- Microsoft Graph sendMail